Step 1: Listing all possible unigram mappings

Imagine we are comparing two sentences:

> *System translation: "The **cat** is playing"*
> *Reference translation: "**Cat** plays outside"*

In the first phase, the system lists all possible ways to match individual words (called "unigrams") between these two sentences.

One word, "**cat**," appears in both sentences. So, we could map the "cat" from the system translation to the "Cat" in the reference translation.

Different rules (modules) are used to determine if two unigrams can be mapped:

1. **Exact module**: Maps unigrams if they are exactly the same. For example, "cat" can map to "cat" but not to "cats" (plural).
2. **Porter stem module**: Maps unigrams after removing word endings. For example, "cat" could also map to "cats" because after removing the plural "s," they match.
3. **Synonym module**: Maps unigrams if they are synonyms. For example, "cat" could map to "feline" because they are synonyms.

Step 2: Selecting the best unigram mappings

Once all possible mappings are listed, the system selects the best set of mappings without any repetition. In other words, each word in the system translation can only be mapped to one word in the reference translation.

If multiple sets of mappings are equally good, the system will choose the one with the **least number of crosses**. What does this mean? Imagine lining up the two sentences:

> *System translation: "The **cat** is playing"*
> *Reference translation: "**Cat** plays outside"*

Now, draw lines between the mapped words (unigrams).
For example: "cat" maps to "Cat"

If these lines cross over each other (like if we were mapping "The" to "outside"), it would be a "cross." The system tries to minimize such crossings.

To formally define a cross, consider positions:
Let's say "The" is at position 1, "cat" at position 2, "is" at position 3, and "playing" at position 4 in the system translation. Similarly, "Cat" is at position 1, "plays" at position 2, and "outside" at position 3 in the reference translation.

If the mappings make two words connect in such a way that their positions "crisscross," the system counts it as a cross.

Step 3: Stage-specific mapping

Each stage focuses on different mapping rules.

- The first stage might map only exact matches (e.g., “cat” to “cat”).
- The second stage could use stem matching (e.g., “playing” to “plays”).
- The third stage could try synonyms (e.g., “feline” to “cat”).

The order of stages matters. For instance, if the system first maps exact matches, it won’t try stemming until the next stage, giving priority to exact matches first.

This process allows for flexibility, as different modules and stages can be configured to suit different needs.

Score Calculation

Step 1: Unigram Precision (P) and Recall (R)

- **Precision (P)** measures how many words (unigrams) in the system translation have correct matches in the reference translation.
- **Recall (R)** measures how well the system captures all the important words from the reference translation.

Let's say we have:

> *System translation: "**The cat** is playing"*
> *Reference translation: "**A cat** plays outside"*

- **Number of unigrams in the system translation:** 4 ("The", "cat", "is", "playing")
- **Number of unigrams in the reference translation:** 4 ("A", "cat", "plays", "outside")

Now, we map the unigrams: "cat" from both sentences match.

- **Unigram Precision (P):** 1 word (cat) out of 4 words in the system translation matches, so:
  P = 1/4 = 0.25
- **Unigram Recall (R):** 1 word (cat) out of 4 words in the reference translation matches, so:
  R = 1/4 = 0.25

Step 2: Fmean Calculation

The Fmean combines Precision and Recall, but gives more importance to Recall (R). It's calculated using the following formula:
Fmean = (10·P·R)/(R + 9·P)

Using the values of P and R from above:

- Fmean=(10·0.25·0.25)/(0.25+9·0.25) = 0.625/2.5 = 0.25

So, the **Fmean** score is 0.25, balancing precision and recall while emphasizing recall more.

Step 3: Penalty for Non-Contiguous Matches

To account for longer sequences of matched words, METEOR applies a penalty based on how well the system translation aligns with the reference translation in terms of longer phrases.

- The system groups all matched words into the **fewest number of contiguous "chunks"**. A chunk is a group of adjacent words in the system translation that maps to adjacent words in the reference translation.
- In our example, we only matched one word ("cat"), so there's only **1 chunk**.

If we had a sentence where several adjacent words matched, they would form a single chunk. For example:

> *System translation: "**The cat plays outside**"*
> *Reference translation: "**A cat plays outside**"*

Here, three adjacent words match: "cat", "plays", "outside", which would be grouped into **1 chunk**.

Now, the **penalty formula** is:

Penalty = 0.5 * (# of chunks/ # of unigrams matched)^3

In our initial example with 1 chunk and 1 unigram matched:
Penalty = $0.5 \cdot (1/1)^3 = 0.5$

This means the penalty for this simple translation would be **0.5**.

Step 4: Final METEOR Score

The final METEOR score is calculated by applying this penalty to the Fmean:

- $\text{METEOR} = \text{Fmean} \cdot (1 - \text{Penalty})$

Using the Fmean of 0.25 and Penalty of 0.5:

- $\text{METEOR} = 0.25 \cdot (1 - 0.5) = 0.25 \cdot 0.5 = 0.125$

Thus, the final METEOR score for this translation pair would be **0.125**. This score reflects both unigram matching (precision and recall) and penalizes if the matches are not contiguous.

Summary

- For each **system translation**, METEOR calculates a score for every reference translation and selects the best score.
- The **overall METEOR score** for the system is then based on aggregate statistics (precision, recall, penalty) across the entire dataset, much like how BLEU works, ensuring a comprehensive evaluation of the system's performance across multiple translations.

Pros of METEOR

1. **Focus on Recall:**
   Unlike BLEU, which emphasizes precision, METEOR puts more weight on recall, making it more sensitive to capturing the full meaning of the reference translation. This allows it to better assess translations that convey the full content, even if not word-for-word.

2. **Synonymy and Stemming:**
   METEOR incorporates synonyms and word stemming (via modules like "WN Synonymy" and "Porter Stem"), which allows it to match semantically related words. For example, "feline" and "cat" or "plays" and "playing" can be considered equivalent, capturing more natural language flexibility than surface-level matches.

3. **Chunk-based Penalty:**
   It penalizes non-contiguous matches through a chunk-based system. This allows METEOR to reward translations that maintain the flow and structure of the reference translation, improving alignment in terms of word order and grammatical coherence.

4. **Higher Correlation with Human Judgment:**
   METEOR has been shown to correlate more closely with human judgments of translation quality compared to BLEU, especially for single-sentence translations, making it more suitable for detailed evaluations.

5. **Handles Multiple Reference Translations:**
   It computes the score for multiple reference translations and selects the best match. This provides flexibility in evaluating translations against different valid ways to phrase the same content.

Cons of METEOR

1. **Complexity**:
   METEOR is more computationally intensive than simpler metrics like BLEU because it requires stemming, synonym matching, and chunking. This makes it slower to compute, especially for large datasets.

2. **Heuristic-based Alignment**:
   The process of selecting the best unigram alignment based on heuristic rules (like minimizing crosses) may not always capture the best semantic alignment, especially for longer or more complex sentences.

3. **Linguistic Dependency**:
   While METEOR can handle synonyms and stemming, it relies on pre-defined linguistic resources (e.g., WordNet for synonyms). These resources can vary in quality across languages, making METEOR less effective for non-English translations or languages with less linguistic support.

4. **Limited to Word-Level Matching**:
   Though METEOR improves over BLEU in handling synonyms and stems, it still operates at the word level. It doesn't fully capture more advanced linguistic structures like syntax or deeper semantics, which limits its ability to evaluate more nuanced translations.

5. **Penalty can be Harsh**:
   The penalty for non-contiguous word alignment can be harsh, reducing the score by up to 50%. This might undervalue translations that are accurate but reorder the sentence differently from the reference in a natural or acceptable way.

6. **Less Effective for Long Documents**:
   METEOR is often better for sentence-level evaluation. For document-level evaluation, the focus on recall and detailed unigram matching can be less suitable, where BLEU or ROUGE might work better.

In summary, METEOR is a more linguistically informed and recall-focused metric that excels at sentence-level evaluations and captures more flexible language use. However, its complexity, dependency on linguistic resources, and potential harsh penalties can be drawbacks in certain contexts.